

Retrieval Optimization: Query Expansion, Reranking, and Recall Metrics

Naive RAG retrieves the top-K chunks most similar to the raw user query. This approach fails when the query is ambiguous, uses different terminology than the document, or requires synthesizing information from multiple non-contiguous sections. Retrieval optimization techniques address these failure modes through query transformation, multi-query expansion, and reranking.

1. Query Rewriting

Query rewriting uses an LLM to transform the user's original query into a cleaner, more retrieval-friendly form before embedding. The rewriter can expand abbreviations, remove conversational filler, and make implicit assumptions explicit. For example, the query 'what did they say about the deadline?' becomes 'What is the project submission deadline mentioned in the document?'. Rewriting consistently improves retrieval precision on conversational or underspecified queries.

2. Multi-Query Expansion

Multi-query expansion generates multiple reformulations of the original query and retrieves chunks for each variant independently. The retrieved sets are then merged and deduplicated. This approach increases recall by covering different phrasings and perspectives of the same information need. A query rewriter might generate three variants: a direct question, a keyword-focused version, and a broader contextual version. Each variant is embedded and used for independent retrieval.

2.1 Deduplication

When multiple query variants retrieve overlapping chunks, deduplication is required before passing context to the LLM. Deduplication is typically performed by chunk ID or content hash. Without deduplication, the same information appears multiple times in the context window, wasting tokens and potentially biasing the LLM toward repeated content.

2.2 Candidate Pool Size

With N query variants each retrieving K chunks, the candidate pool before deduplication contains up to $N \times K$ chunks. For three query variants with $K=8$, the pool contains up to 24 chunks. After deduplication, this pool is passed to a reranker rather than directly to the LLM, as the full 24-chunk context would exceed practical context window limits and degrade generation quality.

3. Reranking

Reranking is a second-stage retrieval step that reorders the candidate chunk pool using a more computationally expensive but higher-accuracy model. While bi-encoder models embed queries and chunks independently (enabling fast ANN search), cross-encoder models process the query and each chunk together, capturing fine-grained interaction signals. Cross-encoders consistently outperform bi-encoders on relevance scoring at the cost of higher latency.

3.1 Cross-Encoder Architecture

A cross-encoder takes a (query, chunk) pair as input and outputs a single relevance score. Unlike bi-encoders, cross-encoders cannot pre-compute chunk representations offline, as the score depends on the specific query. This makes cross-encoders unsuitable as first-stage retrievers but ideal as rerankers over a small candidate pool. The ms-marco-MiniLM series of cross-encoders are commonly used for English text reranking tasks.

3.2 Reranking vs. Vector Similarity

Vector similarity search optimizes for embedding-space proximity, which approximates semantic similarity but does not directly model relevance to a specific query. A cross-encoder reranker directly models query-document relevance, producing more accurate orderings at the top of the ranked list. However, reranking does not add new candidates to the pool; it only reorders existing ones. Recall is therefore bounded by the candidate pool quality before reranking.

4. Recall Metrics

Recall at K (Recall@K) measures what fraction of all relevant information is present in the top K retrieved chunks. It is defined as the number of relevant sentences appearing in the top K chunks divided by the total number of relevant sentences in the document. Recall@K is the primary metric for retrieval evaluation because downstream generation quality is bounded by retrieval recall: information not retrieved cannot appear in the answer.

4.1 Recall@1, Recall@3, Recall@5, Recall@8

Recall@1 measures whether the single most relevant chunk is retrieved. This is typically low for multi-faceted queries. Recall@3 and Recall@5 measure coverage with small context windows. Recall@8 measures coverage with a larger context. A well-optimized retrieval pipeline should maximize Recall@K at the K value that corresponds to the number of chunks passed to the LLM. For a pipeline passing 8 chunks to the generator, Recall@8 is the operationally relevant metric.

4.2 Precision vs. Recall Tradeoff

Increasing K improves recall but reduces precision: more irrelevant chunks enter the context window, adding noise and consuming tokens. The optimal K balances recall coverage against context dilution.

Reranking helps this tradeoff by improving the precision of the top K results without reducing recall, assuming the candidate pool before reranking is large enough to contain all relevant chunks.

5. Hybrid Search

Hybrid search combines dense vector similarity with sparse keyword search (typically BM25). Dense retrieval excels at semantic matching, while BM25 excels at exact term matching. Combining both with a weighted fusion score (such as Reciprocal Rank Fusion) improves recall on queries involving product names, error codes, or other exact identifiers that may not cluster semantically with their relevant documents.